Kshitiz Gupta

Intermediate SQL Assignment

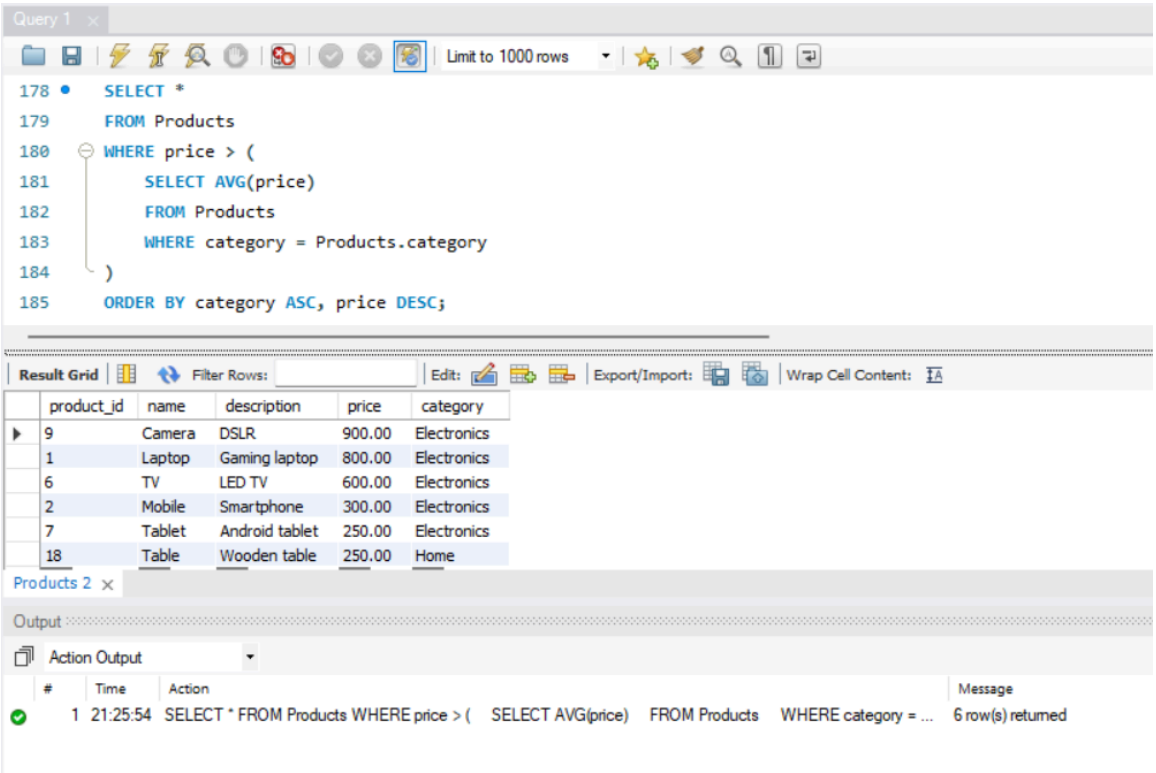
Note : Created separate tables when required instead of adding new columns in Product, Order and Customer table for better understanding.

a. Products higher than category average

Query:

```
SELECT *
FROM Products
WHERE price > (
   SELECT AVG(price)
   FROM Products
   WHERE category = Products.category
)
ORDER BY category ASC, price DESC;
```

Screenshot:



b. Products more expensive than Furniture category

Query:

```
SELECT *
FROM Products
WHERE price > (
   SELECT MAX(price)
   FROM Products
   WHERE category = 'Furniture'
);
```

Screenshot:

```
351 • INSERT INTO Products (name, description, price, category) VALUES
352   ('Sofa','Comfort sofa',300,'Furniture'),
353   ('Table','Wood table',200,'Furniture'),
354   ('Chair','Office chair',150,'Furniture');
355
356 • SELECT *
357   FROM Products
358   WHERE price > (
359       SELECT MAX(price)
360       FROM Products
361       WHERE category = 'Furniture'
362   );
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	name	description	price	category
1	Laptop	Gaming laptop	800.00	Electronics
6	TV	LED TV	600.00	Electronics
9	Camera	DSLR	900.00	Electronics
NULL	NULL	NULL	NULL	NULL

Products 28

Output

Action Output

#	Time	Action	Message
1	22:36:07	SELECT * FROM Products WHERE price > (SELECT MAX(price) FROM Products WHERE category = '...	3 row(s) returned

c. Stored Procedure (Discount)

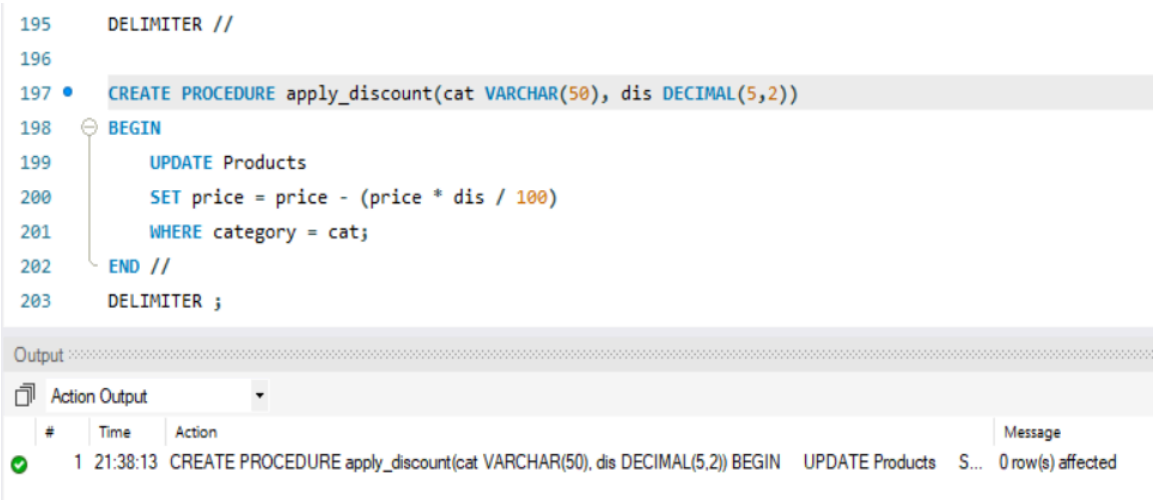
Query:

```
DELIMITER //

CREATE PROCEDURE apply_discount(cat VARCHAR(50), dis DECIMAL(5,2))
BEGIN
   UPDATE Products
   SET price = price - (price * dis / 100)
   WHERE category = cat;
END //

DELIMITER ;
```

Screenshot:



d. Trigger (Total Spending)

Query:

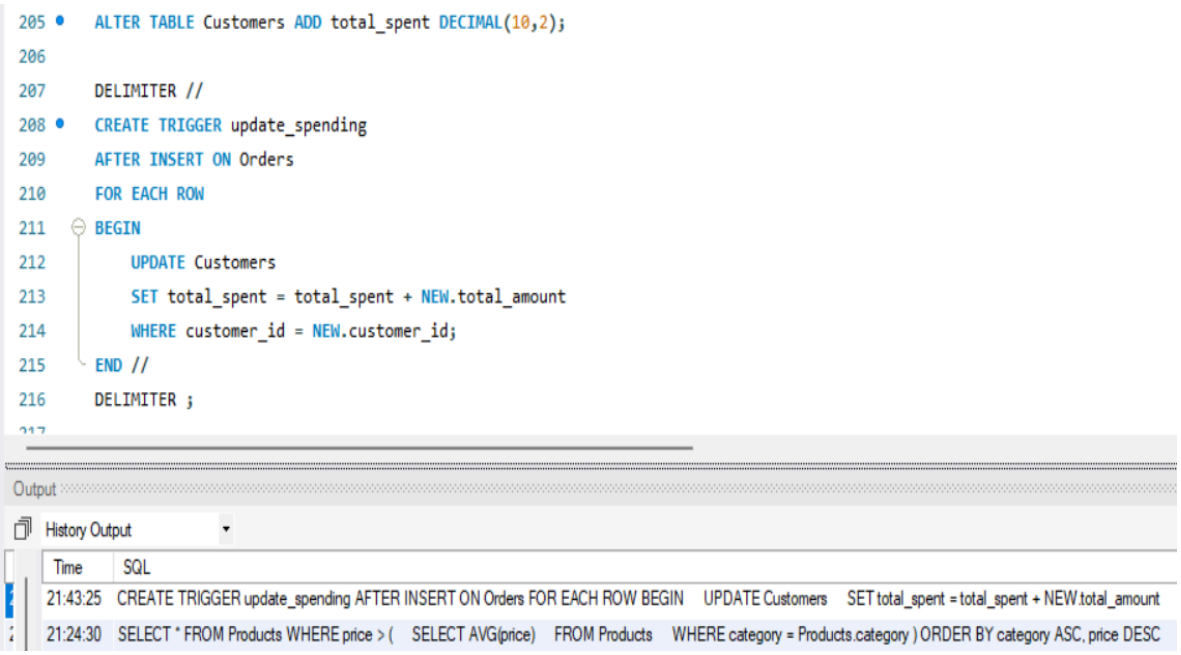
```
ALTER TABLE Customers ADD total_spent DECIMAL(10,2);

DELIMITER //

CREATE TRIGGER update_spending
AFTER INSERT ON Orders
FOR EACH ROW
BEGIN
   UPDATE Customers
   SET total_spent = total_spent + NEW.total_amount
   WHERE customer_id = NEW.customer_id;
END //

DELIMITER ;
```

Screenshot:

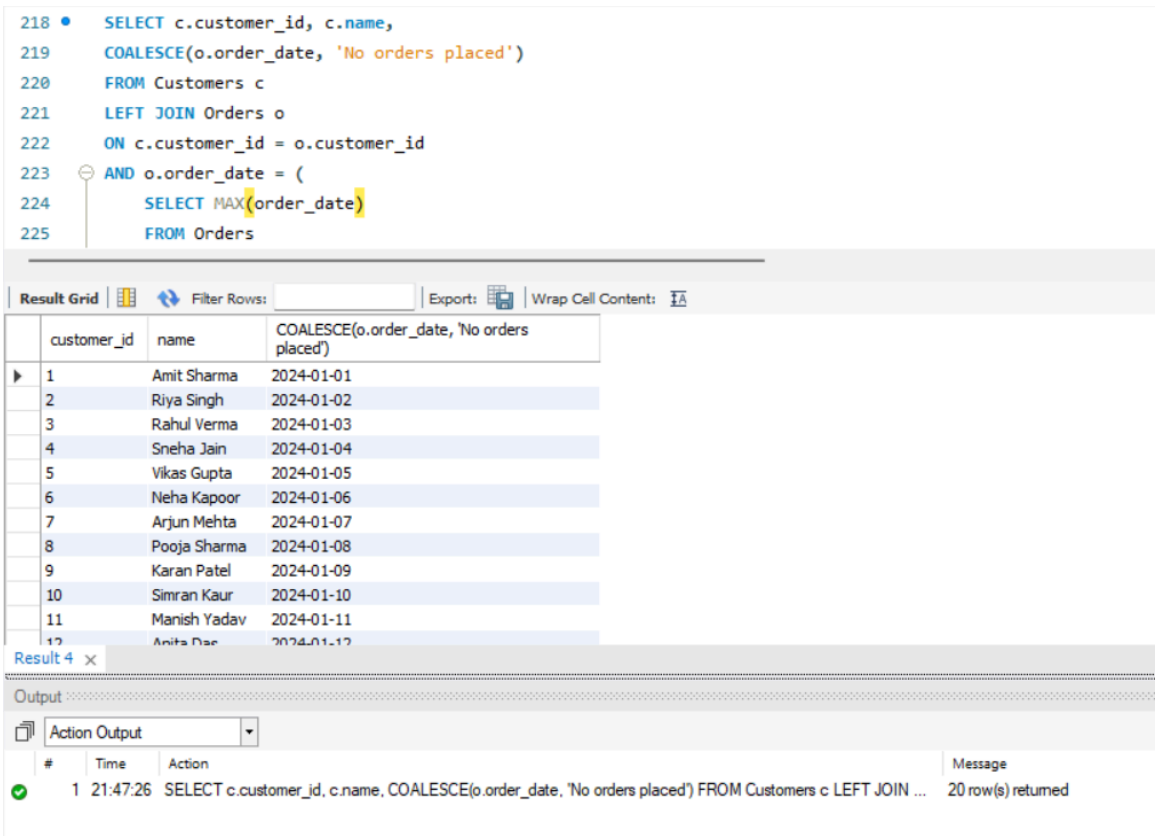


e. Customers with latest order

Query:

```
SELECT c.customer_id, c.name,
COALESCE(o.order_date, 'No orders placed')
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
AND o.order_date = (
   SELECT MAX(order_date)
   FROM Orders
   WHERE customer_id = c.customer_id
);
```

Screenshot:



```sql
SELECT c.customer_id, c.name,
COALESCE(o.order_date, 'No orders placed')
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
AND o.order_date = (
    SELECT MAX(order_date)
    FROM Orders
    WHERE customer_id = c.customer_id
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name	COALESCE(o.order_date, 'No orders placed')
11	Manish Yadav	2024-01-11
12	Anita Das	2024-01-12
13	Rohit Roy	2024-01-13
14	Meera Nair	2024-01-14
15	Suresh Kumar	2024-01-15
16	Priya Iyer	2024-01-16
17	Deepak Singh	2024-01-17
18	Nikita Shah	2024-01-18
19	Raj Malhotra	2024-01-19
20	Alok Pandey	2024-01-20

Result 4

Output

Action Output

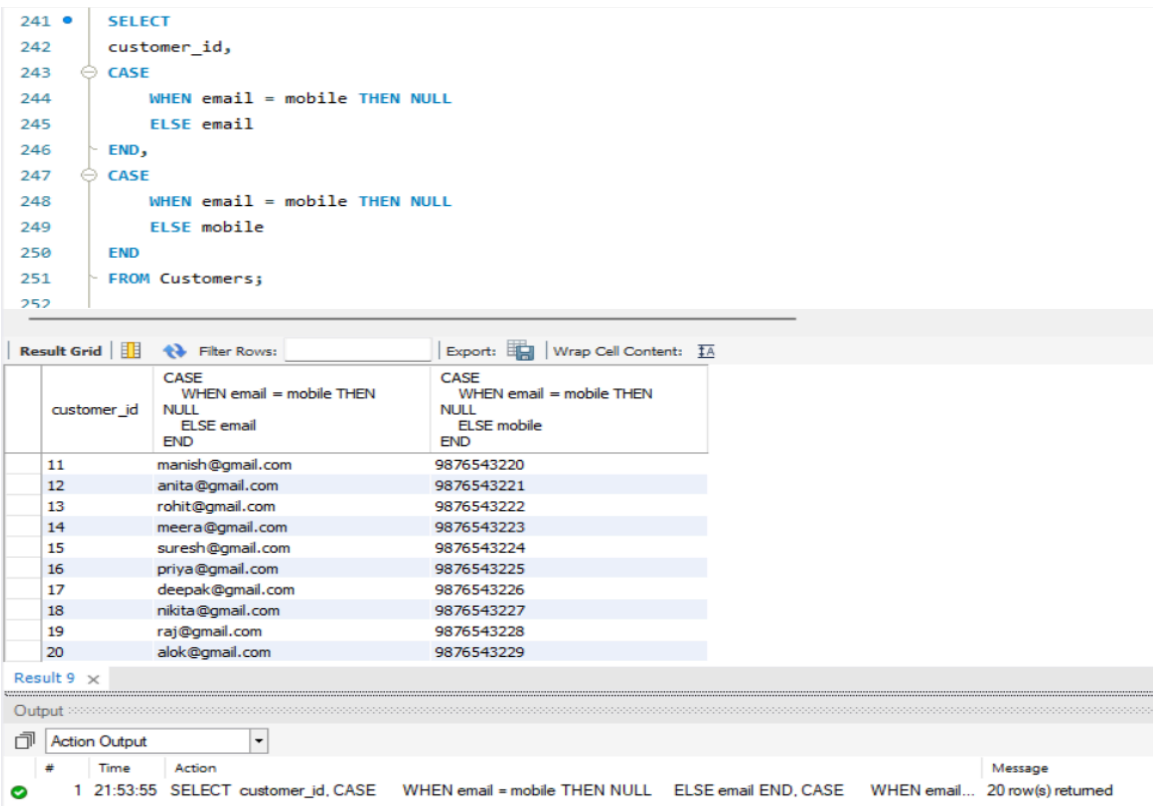
#	Time	Action	Message
1	21:47:26	SELECT c.customer_id, c.name, COALESCE(o.order_date, 'No orders placed') FROM Customers c LEFT JOIN ...	20 row(s) returned

f. Email and mobile same handling

Query:

```
SELECT
customer_id,
CASE
   WHEN email = mobile THEN NULL
   ELSE email
END,
CASE
   WHEN email = mobile THEN NULL
   ELSE mobile
END
FROM Customers;
```

Screenshot:

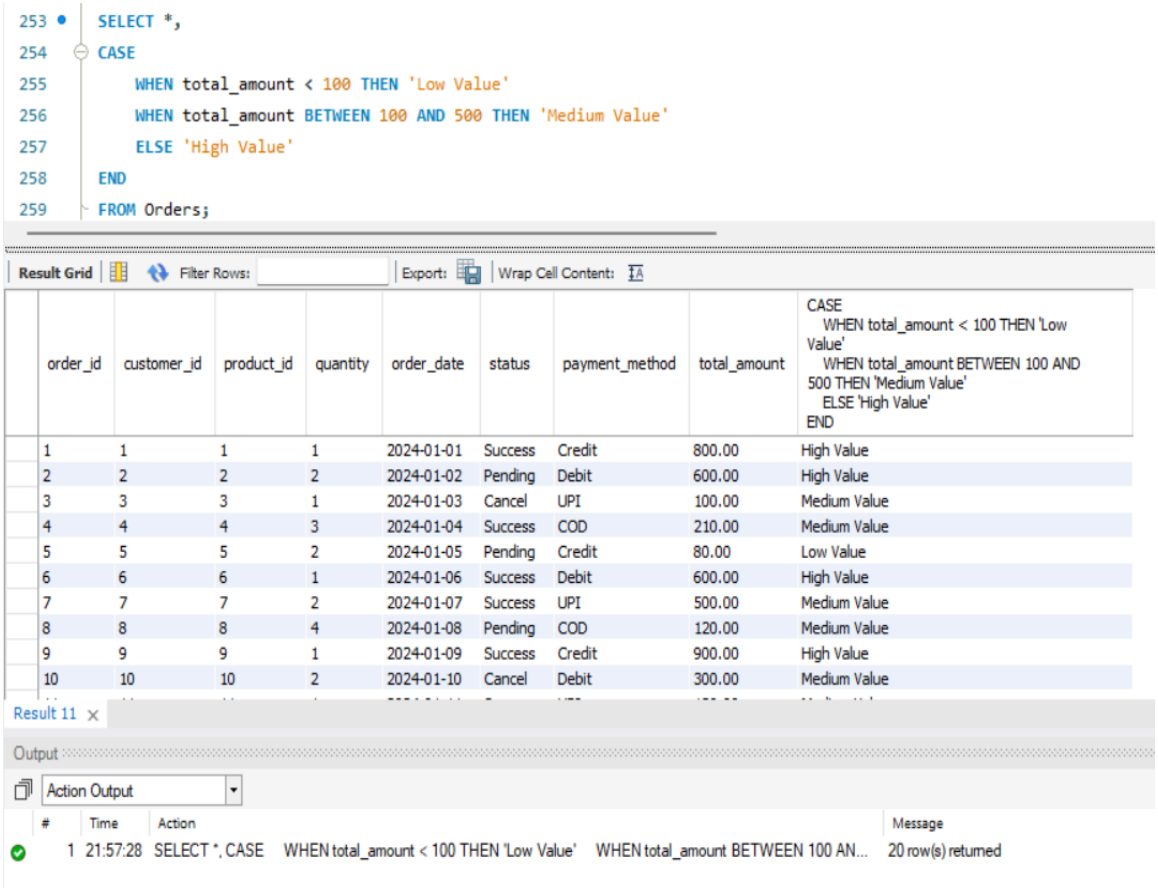


g. Categorize orders

Query:

```
SELECT *,
CASE
   WHEN total_amount < 100 THEN 'Low Value'
   WHEN total_amount BETWEEN 100 AND 500 THEN 'Medium Value'
   ELSE 'High Value'
END
FROM Orders;
```

Screenshot:-



```
253 • SELECT *,
254   CASE
255       WHEN total_amount < 100 THEN 'Low Value'
256       WHEN total_amount BETWEEN 100 AND 500 THEN 'Medium Value'
257       ELSE 'High Value'
258   END
259   FROM Orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	customer_id	product_id	quantity	order_date	status	payment_method	total_amount	CASE WHEN total_amount < 100 THEN 'Low Value' WHEN total_amount BETWEEN 100 AND 500 THEN 'Medium Value' ELSE 'High Value' END
1	1	1	1	2024-01-01	Success	Credit	800.00	High Value
2	2	2	2	2024-01-02	Pending	Debit	600.00	High Value
3	3	3	1	2024-01-03	Cancel	UPI	100.00	Medium Value
4	4	4	3	2024-01-04	Success	COD	210.00	Medium Value
5	5	5	2	2024-01-05	Pending	Credit	80.00	Low Value
6	6	6	1	2024-01-06	Success	Debit	600.00	High Value
7	7	7	2	2024-01-07	Success	UPI	500.00	Medium Value
8	8	8	4	2024-01-08	Pending	COD	120.00	Medium Value
9	9	9	1	2024-01-09	Success	Credit	900.00	High Value
10	10	10	2	2024-01-10	Cancel	Debit	300.00	Medium Value

Result 11

Output

Action Output

#	Time	Action	Message
1	21:57:28	SELECT *, CASE WHEN total_amount < 100 THEN 'Low Value' WHEN total_amount BETWEEN 100 AN...	20 row(s) returned

```sql
SELECT *,
CASE
    WHEN total_amount < 100 THEN 'Low Value'
    WHEN total_amount BETWEEN 100 AND 500 THEN 'Medium Value'
    ELSE 'High Value'
END
FROM Orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	customer_id	product_id	quantity	order_date	status	payment_method	total_amount	CASE WHEN total_amount < 100 THEN 'Low Value' WHEN total_amount BETWEEN 100 AND 500 THEN 'Medium Value' ELSE 'High Value' END
11	11	11	1	2024-01-11	Success	UPI	120.00	Medium Value
12	12	12	3	2024-01-12	Pending	COD	120.00	Medium Value
13	13	13	2	2024-01-13	Success	Credit	120.00	Medium Value
14	14	14	1	2024-01-14	Success	Debit	200.00	Medium Value
15	15	15	2	2024-01-15	Pending	UPI	160.00	Medium Value
16	16	16	5	2024-01-16	Success	COD	100.00	Medium Value
17	17	17	1	2024-01-17	Cancel	Credit	150.00	Medium Value
18	18	18	2	2024-01-18	Success	Debit	500.00	Medium Value
19	19	19	1	2024-01-19	Pending	UPI	90.00	Low Value
20	20	20	3	2024-01-20	Success	COD	135.00	Medium Value

Result 11

Output

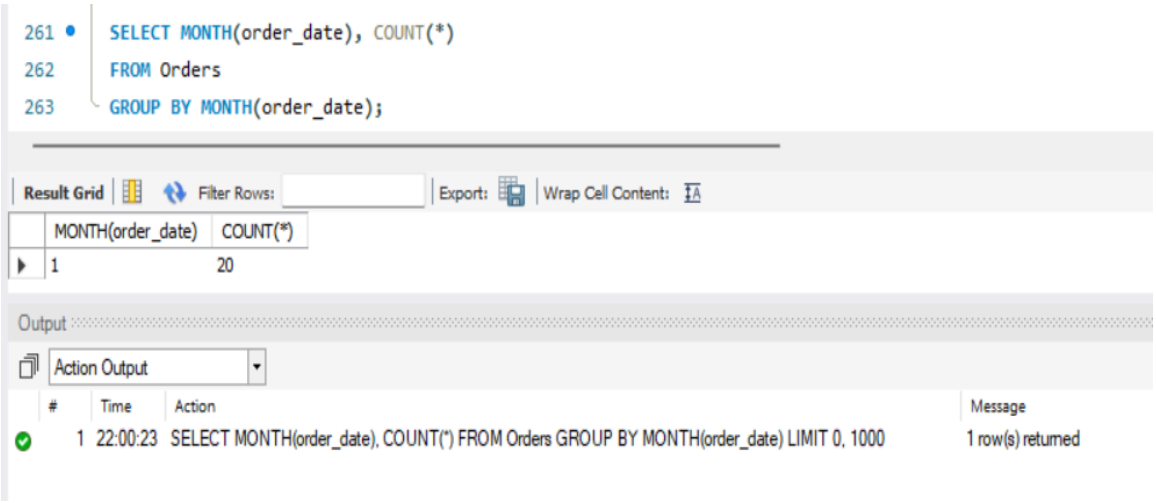
Action Output

#	Time	Action	Message
1	21:57:28	SELECT *, CASE WHEN total_amount < 100 THEN 'Low Value' WHEN total_amount BETWEEN 100 AN...	20 row(s) returned

h. Orders per month

Query:
SELECT MONTH(order_date), COUNT(*)
FROM Orders
GROUP BY MONTH(order_date);

Screenshot:



i. Replace 0 with NULL

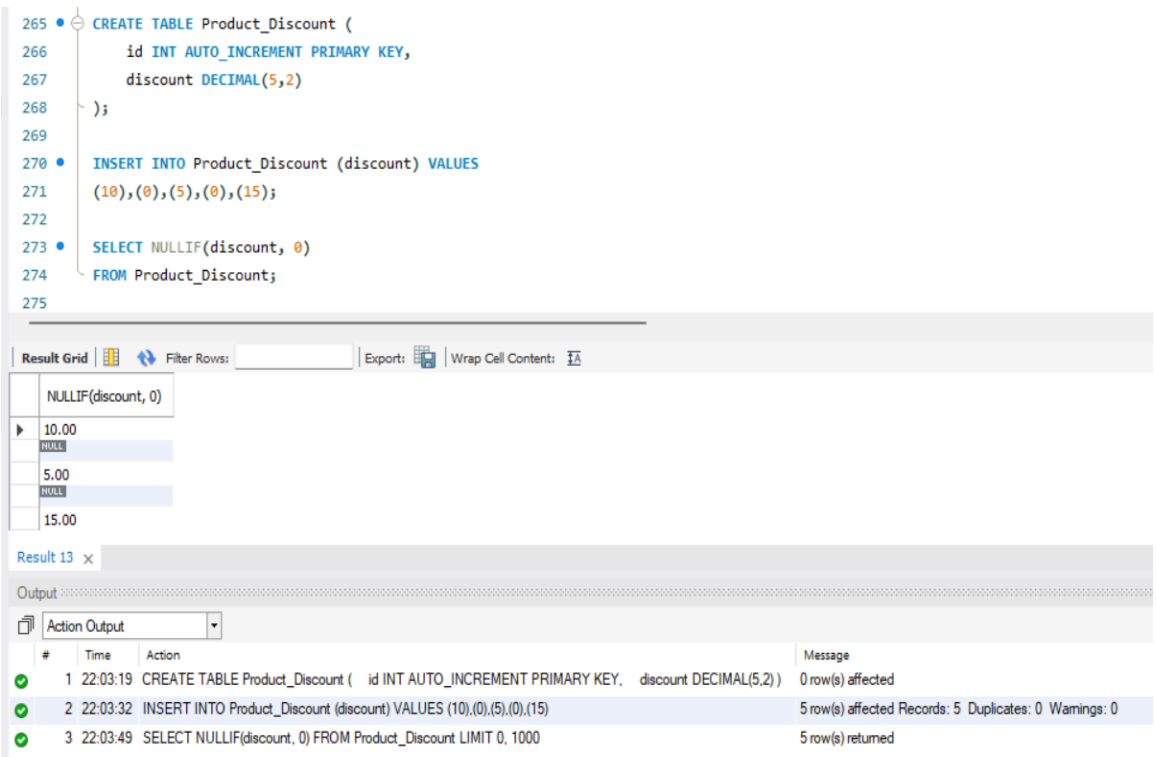
Query:

```
CREATE TABLE Product_Discount (
   id INT AUTO_INCREMENT PRIMARY KEY,
   discount DECIMAL(5,2)
);

INSERT INTO Product_Discount (discount) VALUES
(10),(0),(5),(0),(15);

SELECT NULLIF(discount, 0)
FROM Product_Discount;
```

Screenshot:



j. Safe division using NULLIF

Query:
SELECT total_amount / NULLIF(quantity, 0)
FROM Orders;

Screenshot:

```
276 •  SELECT total_amount / NULLIF(quantity, 0)
277    FROM Orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_amount / NULLIF(quantity, 0)
800.000000
300.000000
100.000000
70.000000
40.000000
600.000000
250.000000
30.000000
900.000000
150.000000
120.000000
40.000000
60.000000
200.000000
80.000000
20.000000
150.000000
250.000000
90.000000
45.000000

Result 15

Output

Action Output

#	Time	Action	Message
1	22:08:36	SELECT total_amount / NULLIF(quantity, 0) FROM Orders LIMIT 0, 1000	20 row(s) returned

k. Count non-zero values

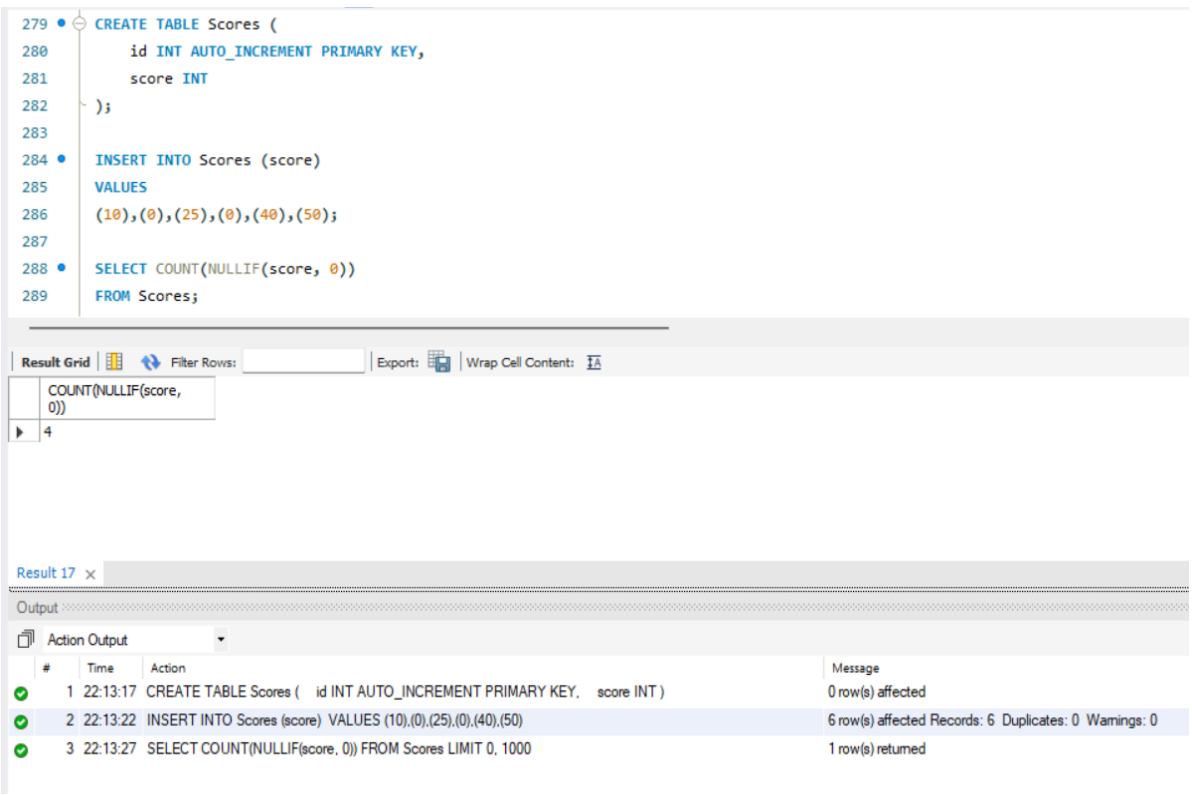
Query:

```
CREATE TABLE Scores (
   id INT AUTO_INCREMENT PRIMARY KEY,
   score INT
);

INSERT INTO Scores (score) VALUES
(10),(0),(25),(0),(40),(50);

SELECT COUNT(NULLIF(score, 0))
FROM Scores;
```

Screenshot:



l. Average salary excluding 0

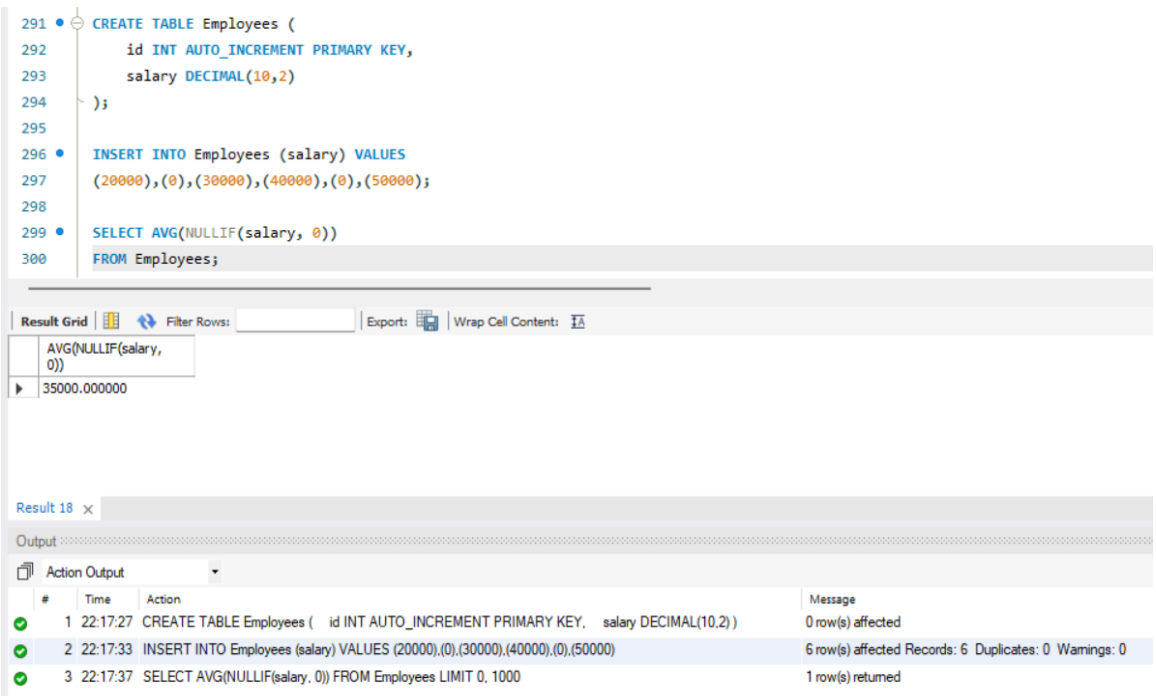
Query:

```
CREATE TABLE Employees (
   id INT AUTO_INCREMENT PRIMARY KEY,
   salary DECIMAL(10,2)
);

INSERT INTO Employees (salary) VALUES
(20000),(0),(30000),(40000),(0),(50000);

SELECT AVG(NULLIF(salary, 0))
FROM Employees;
```

Screenshot:



m. Percentage contribution

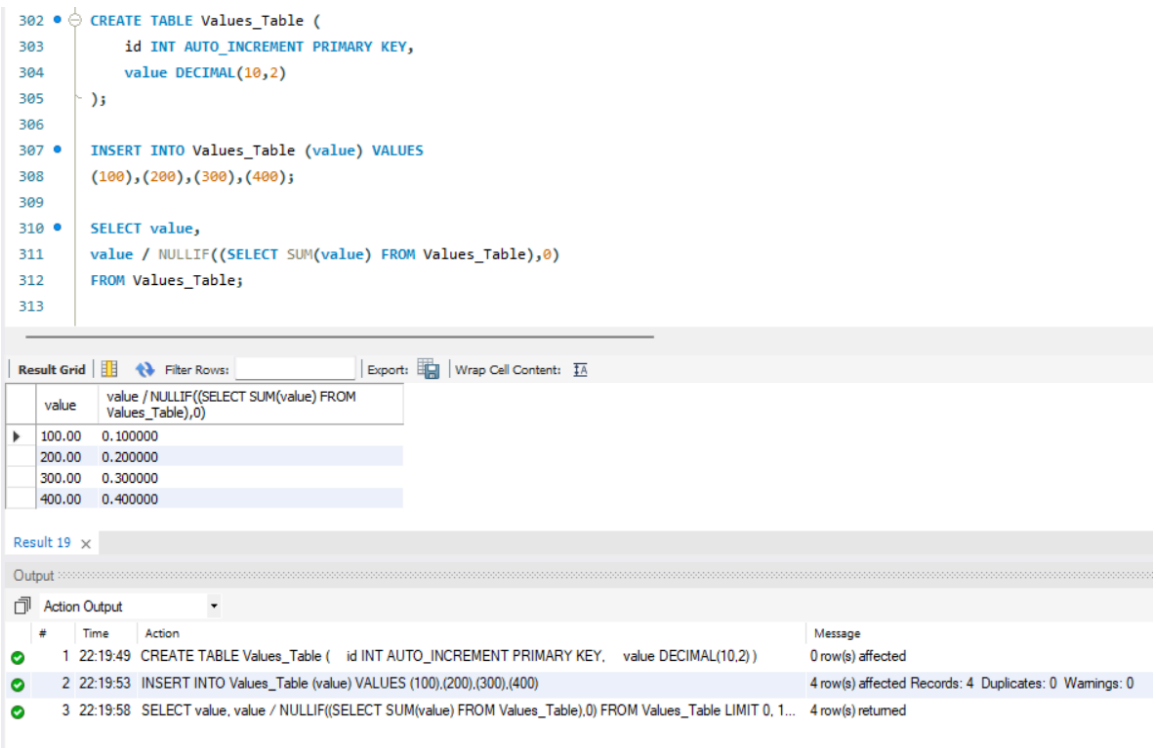
Query:

```
CREATE TABLE Values_Table (
   id INT AUTO_INCREMENT PRIMARY KEY,
   value DECIMAL(10,2)
);

INSERT INTO Values_Table (value) VALUES
(100),(200),(300),(400);

SELECT value,
value / NULLIF((SELECT SUM(value) FROM Values_Table),0)
FROM Values_Table;
```

Screenshot:



n. Empty string to NULL

Query:
SELECT NULLIF(email, '')
FROM Customers;

Screenshot:

```
314 •  SELECT NULLIF(email, '')
315    FROM Customers;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

NULLIF(email, '')
alok@gmail.com
amit@gmail.com
anita@gmail.com
arjun@gmail.com
deepak@gmail.com
karan@gmail.com
manish@gmail.com
meera@gmail.com
neha@gmail.com
nikita@gmail.com
pooja@gmail.com
priya@gmail.com
rahul@gmail.com
raj@gmail.com
riya@gmail.com
rohit@gmail.com
simran@gmail.com
sneha@gmail.com
suresh@gmail.com
vikas@gmail.com

Result 21

Output

Action Output

#	Time	Action	Message
1	22:21:36	SELECT NULLIF(email, '') FROM Customers LIMIT 0, 1000	20 row(s) returned

o. start_date = end_date

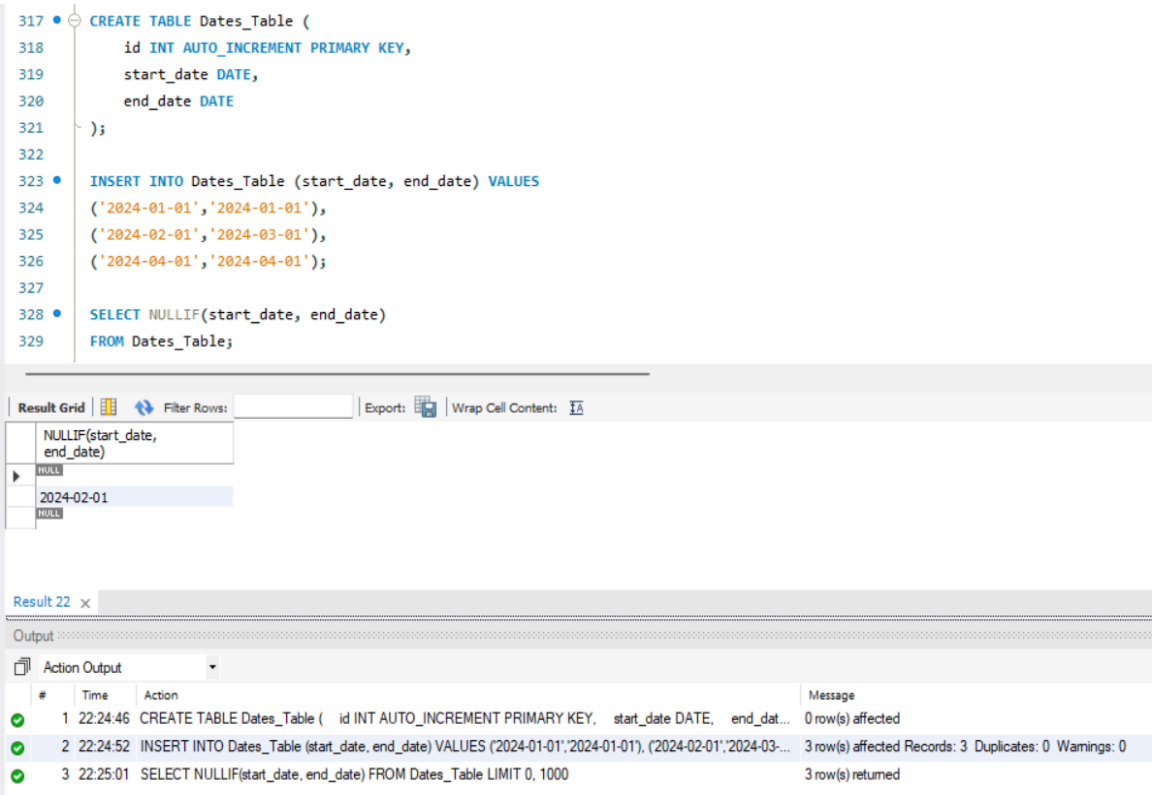
Query:

```
CREATE TABLE Dates_Table (
   id INT AUTO_INCREMENT PRIMARY KEY,
   start_date DATE,
   end_date DATE
);

INSERT INTO Dates_Table (start_date, end_date) VALUES
('2024-01-01','2024-01-01'),
('2024-02-01','2024-03-01'),
('2024-04-01','2024-04-01');

SELECT NULLIF(start_date, end_date)
FROM Dates_Table;
```

Screenshot:

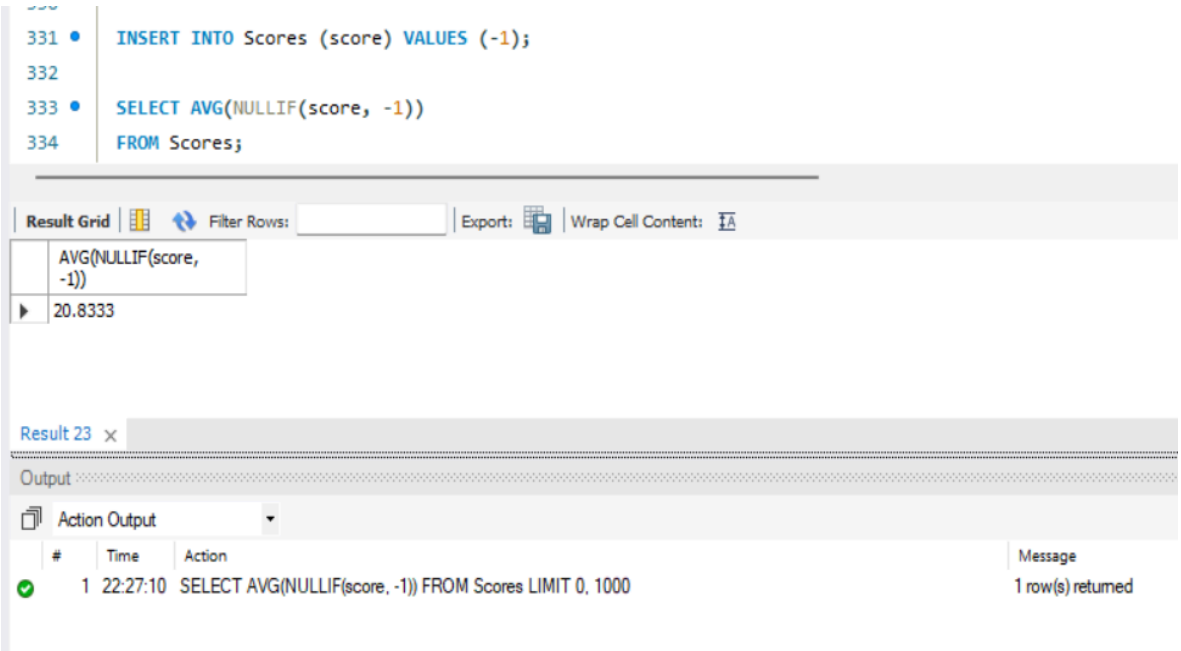


p. Ignore -1 in average

Query:
INSERT INTO Scores (score) VALUES (-1);

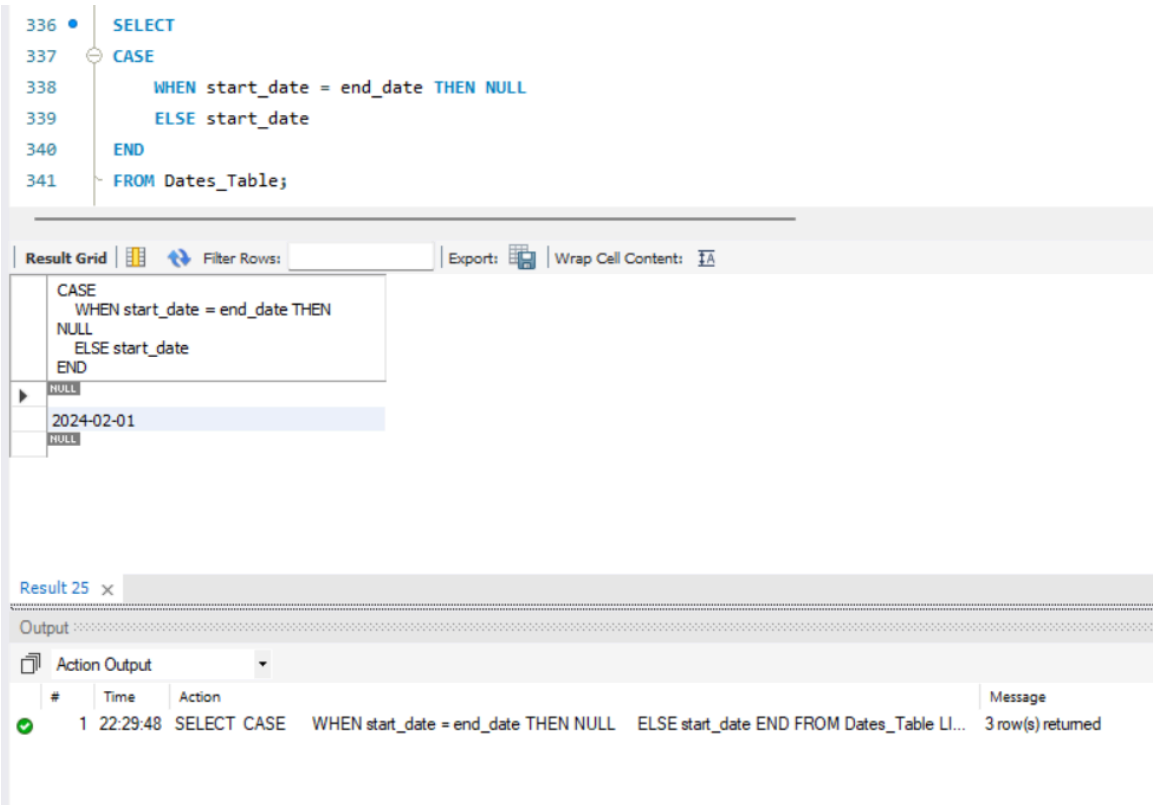
SELECT AVG(NULLIF(score, -1))
FROM Scores;

Screenshot:



q. Replace value when columns match

Query:

```
SELECT
CASE
   WHEN start_date = end_date THEN NULL
   ELSE start_date
END
FROM Dates_Table;
```

Screenshot: